

Signal Processing in Practice

Assignment: Implementation of Linear Prediction and Spectral Transform LP

Prof. Sriram Ganapathy

Tejash More (27077)

February 23, 2026

1 Methodology

1.1 Linear Prediction (LP) and The Source-Filter Model

The fundamental premise of this implementation relies on the source-filter model of speech production. A speech signal $s[n]$ is modeled as the output of a linear, time-varying filter (representing the vocal tract) excited by a source signal (representing the vocal cords or turbulent airflow).

Linear Prediction estimates the current sample of a signal as a linear combination of its past N samples:

$$\hat{x}[n] = \sum_{k=1}^N a_k x[n-k]$$

where a_k are the linear prediction coefficients (LPCs) and N is the prediction order. The parameters a_k are determined by minimizing the mean squared prediction error, leading to the Yule-Walker equations, which are solved using the autocorrelation sequence of the short-term frames.

The residual signal, or prediction error $e[n]$, is computed as:

$$e[n] = x[n] - \hat{x}[n]$$

This residual acts as an approximation of the original excitation source.

1.2 Spectral Transform Linear Prediction (STLP)

Standard LP minimizes a spectral matching measure that inherently biases the fit towards the spectral peaks (formants) while poorly approximating spectral valleys. STLP addresses this by compressing the dynamic range of the Power Spectral Density (PSD) before applying the autoregressive (AR) modeling.

The transformed PSD is given by $S_{xx}(e^{j\omega})^{\frac{1}{q}}$, where q is a positive integer. The inverse Discrete-Time Fourier Transform (DTFT) of this compressed spectrum yields a modified autocorrelation sequence. Standard LP is applied to this modified sequence to derive an AR model of the spectrum, $\hat{S}_{xx}(e^{j\omega})$. The final approximated spectrum is then expanded back:

$$\hat{S}_{xx}(e^{j\omega})^q \approx S_{xx}(e^{j\omega})$$

2 Experimentation Setup

The experiments were conducted with the following parameters based on the assignment specifications:

- **Framing:** Short-Time Fourier Transform (STFT) windows of 25 ms duration.
- **Shift:** 10 ms hop size between consecutive frames.
- **Windowing Function:** Hamming window to mitigate spectral leakage at the frame boundaries.
- **Variables Investigated:** The LP order $N \in \{2, 4, 6, 8, 10, 12, 14, 16, 20, 25, 30\}$ and the STLP root parameter $q \in \{1, 2, 3\}$.

3 Results and Observations

3.1 Part 5(a): LP Residual Energy

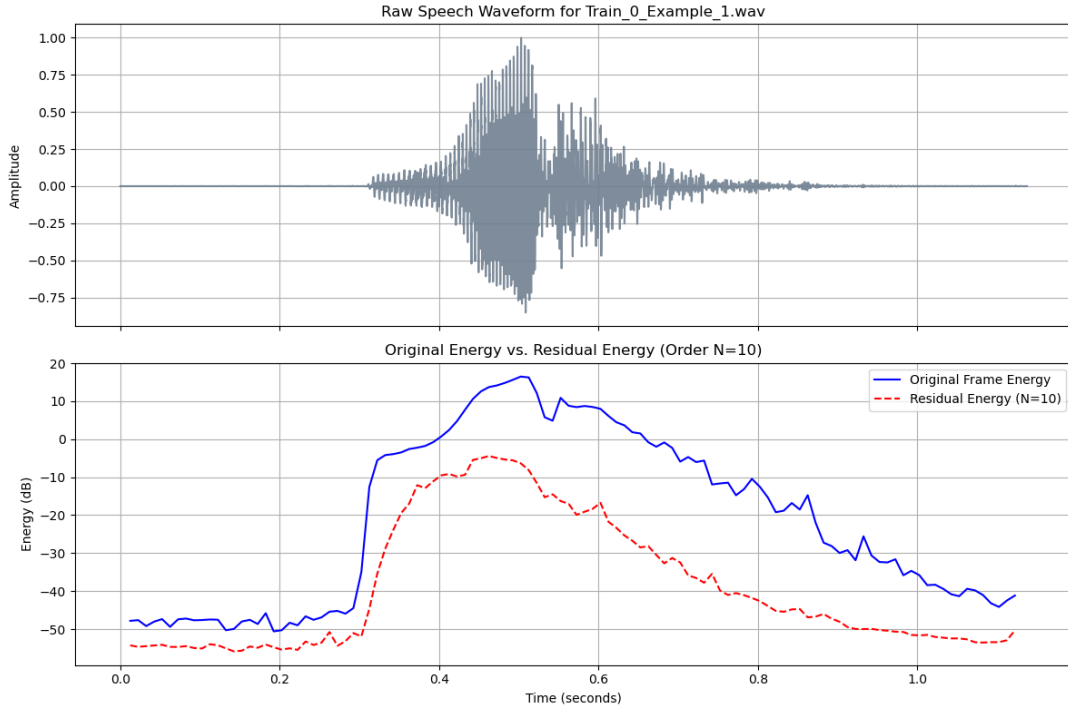


Figure 1: Original Energy vs Residual energy for Train_0_Example_1.wav for $N = 10$

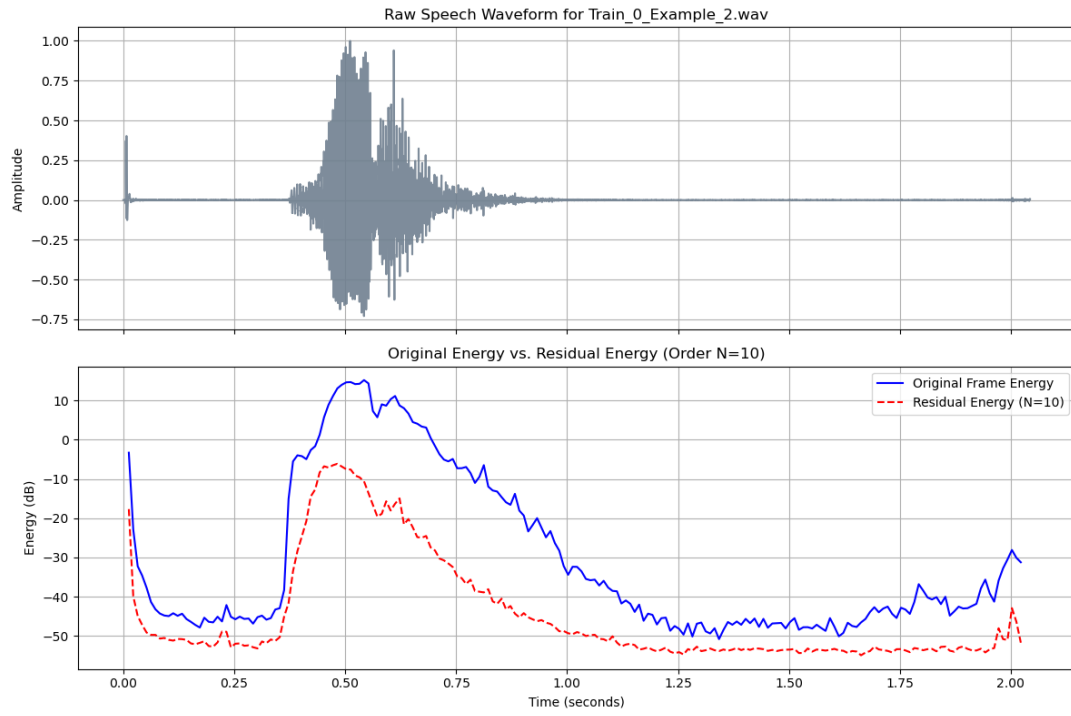


Figure 2: Original Energy vs Residual energy for Train_0_Example_2.wav for $N = 10$

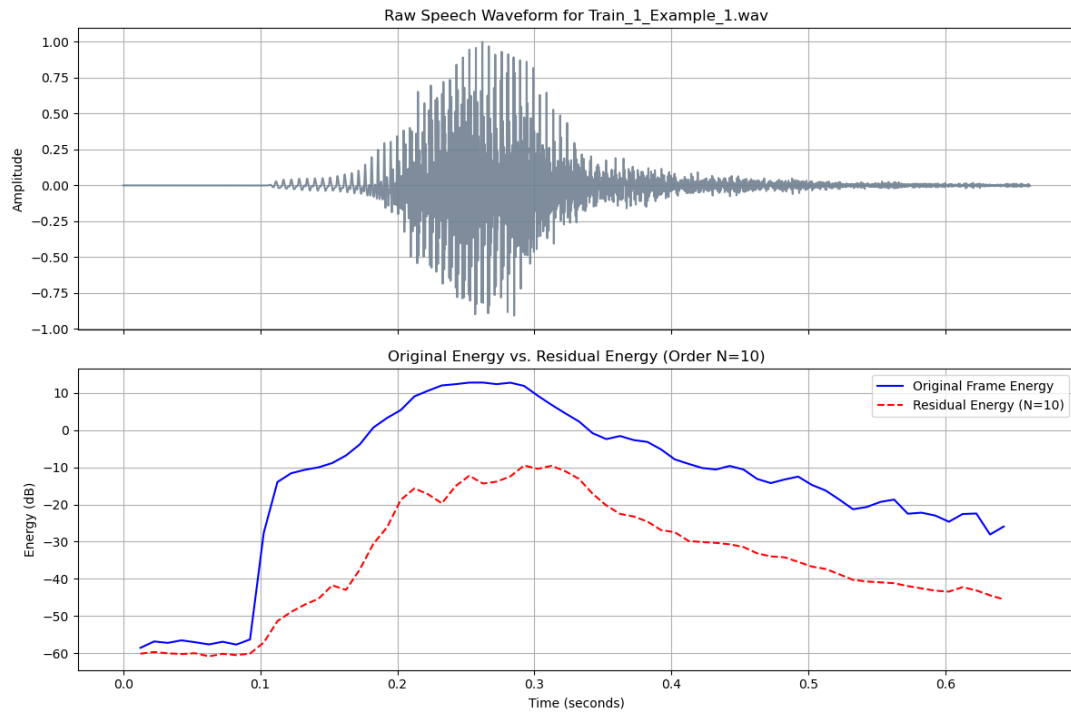


Figure 3: Original Energy vs Residual energy for Train_1_Example_1.wav for $N = 10$

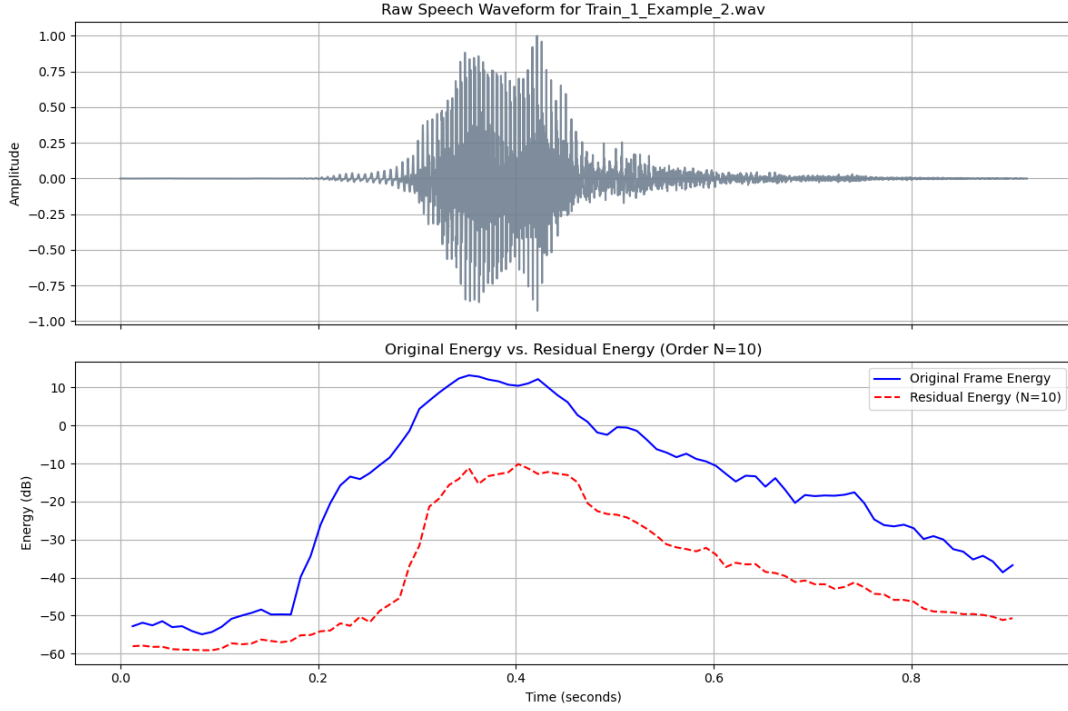
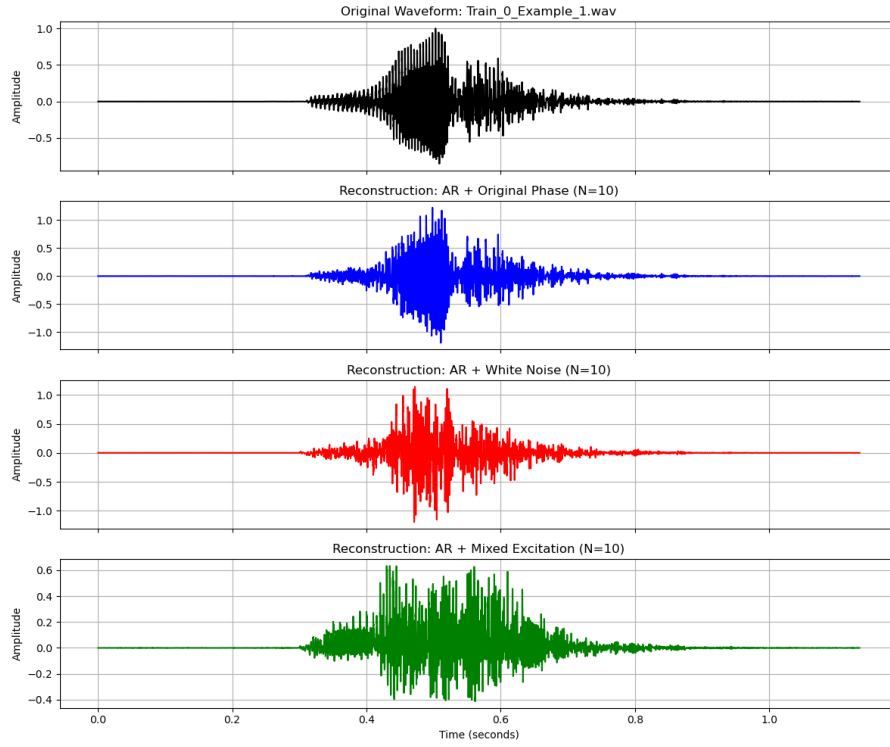


Figure 4: Original Energy vs Residual energy for Train_1_Example_2.wav for $N = 10$

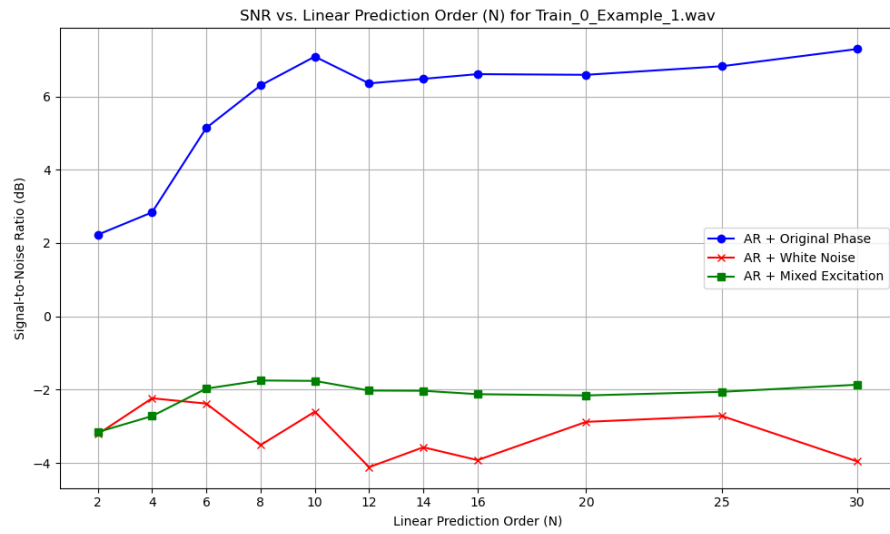
Observation & Analysis: Figure 2 through Figure 4 illustrate the residual energy across consecutive STFT frames. The energy of the residual signal is highly correlated with the underlying phonetic content.

- **Voiced Frames:** During voiced speech (e.g., vowels), the vocal cords produce quasi-periodic glottal pulses. The inverse LP filter removes the vocal tract formants, leaving behind these impulse-like excitations. Consequently, the residual energy exhibits distinct, high-amplitude peaks.
- **Unvoiced Frames:** For unvoiced consonants, the excitation is essentially turbulent noise (frication). The resulting residual resembles random white noise with a noticeably lower and flatter energy profile compared to voiced segments.

3.2 Part 5(b): Signal Reconstruction and Saturation of N

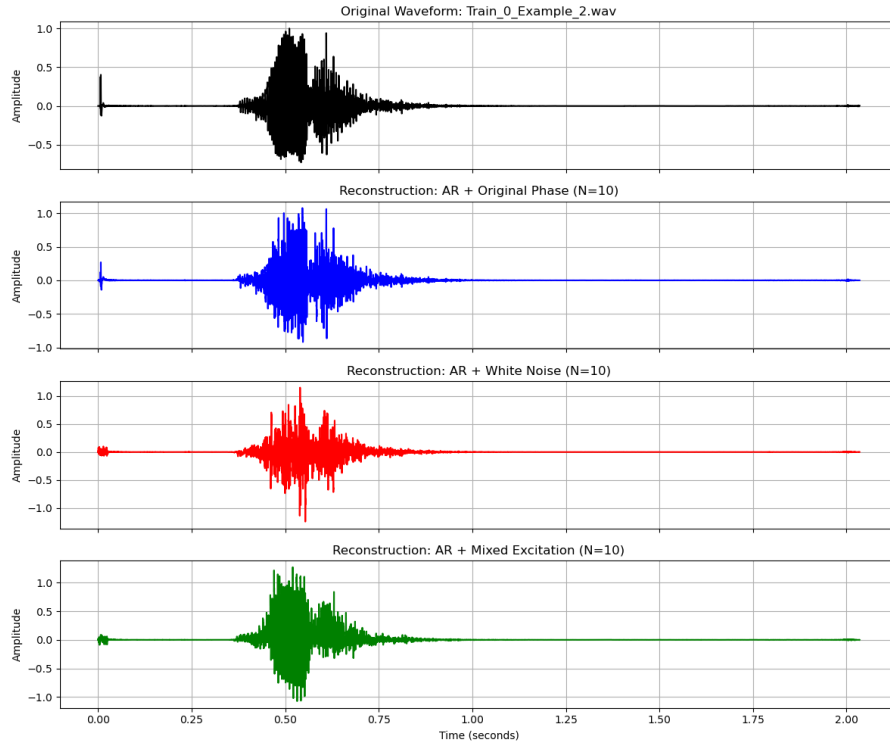


(a) Waveform Comparison for $N = 10$

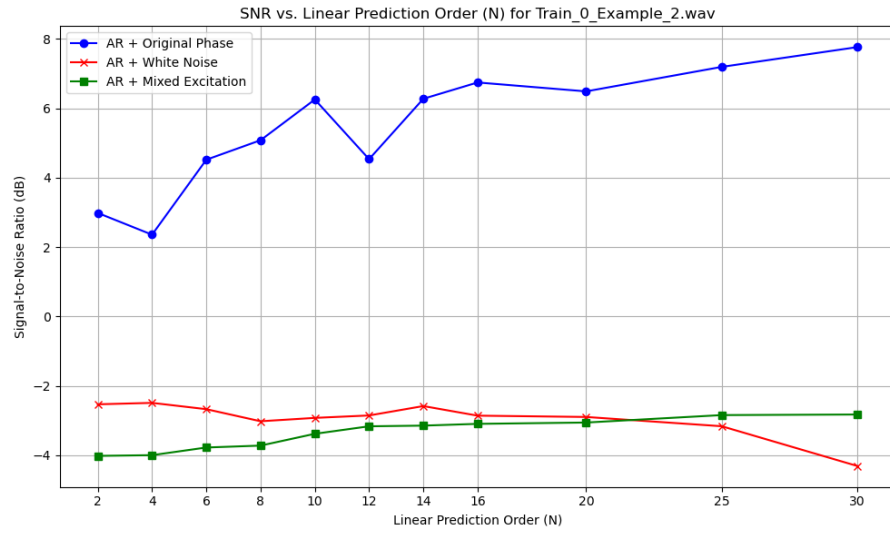


(b) SNR vs Level Order (N)

Figure 5: Train_0_Example_1.wav

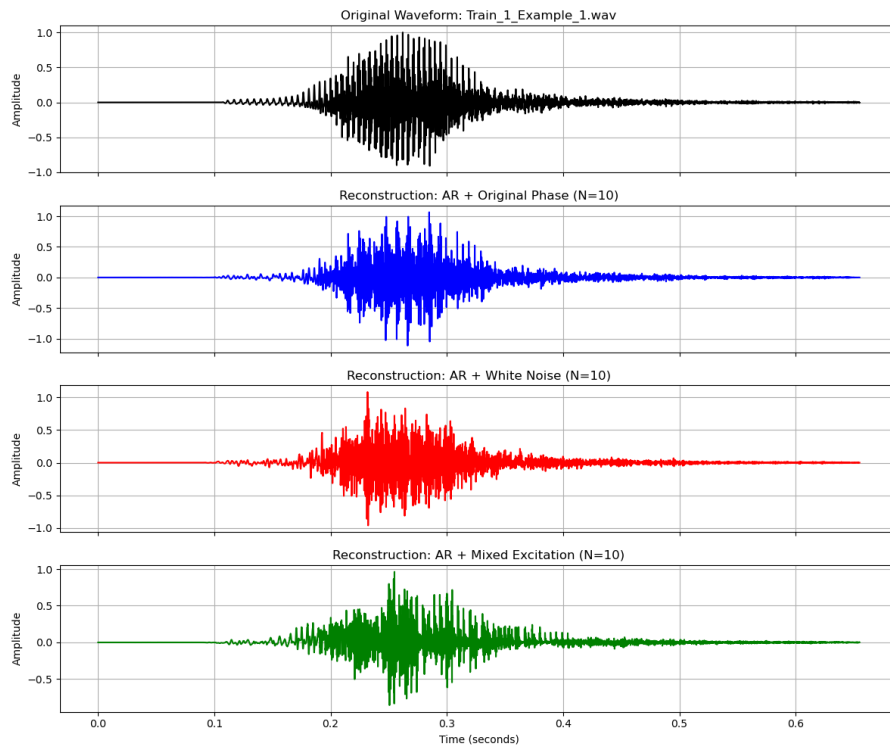


(a) Waveform Comparison for $N = 10$

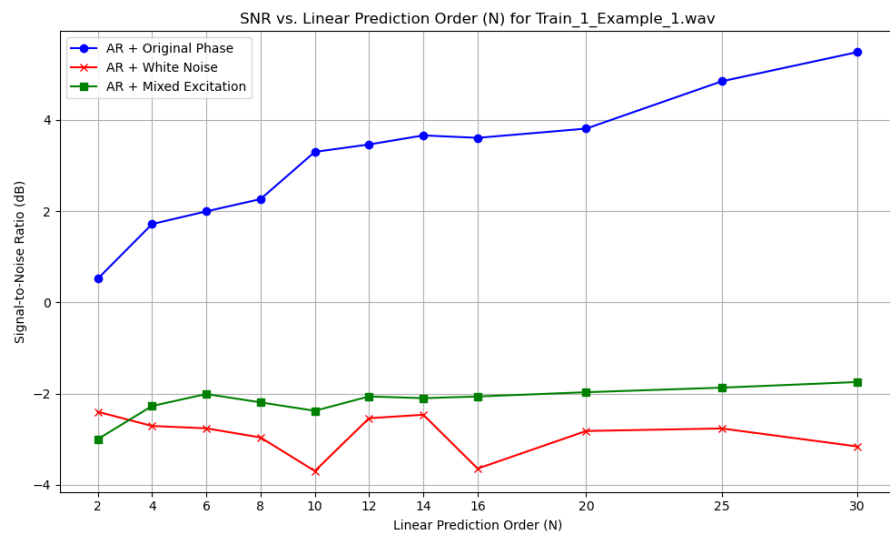


(b) SNR vs Level Order (N)

Figure 6: Train_0_Example_2.wav

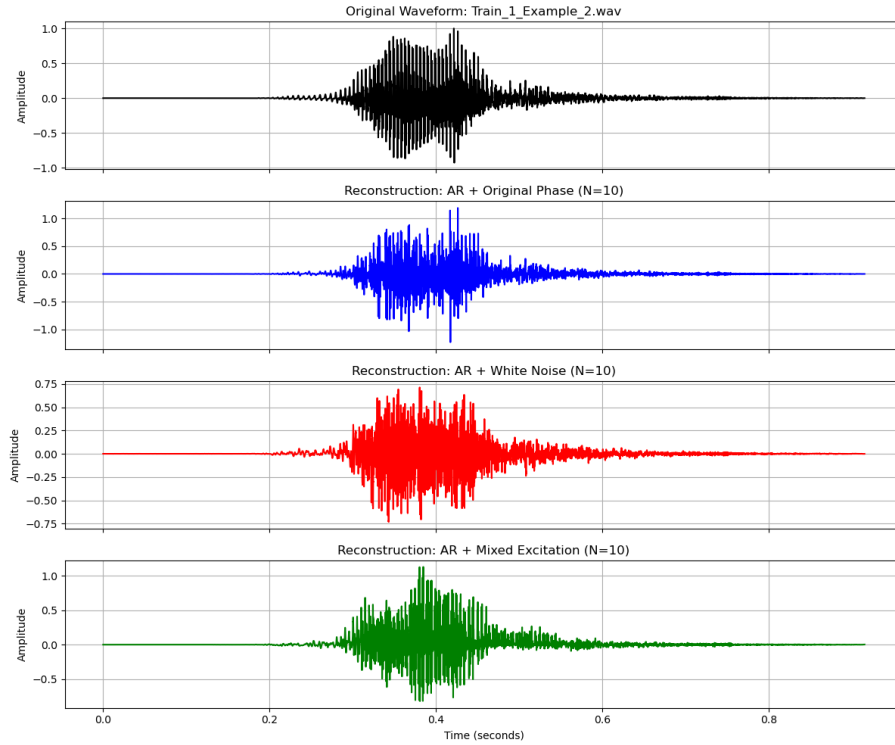


(a) Waveform Comparison for $N = 10$

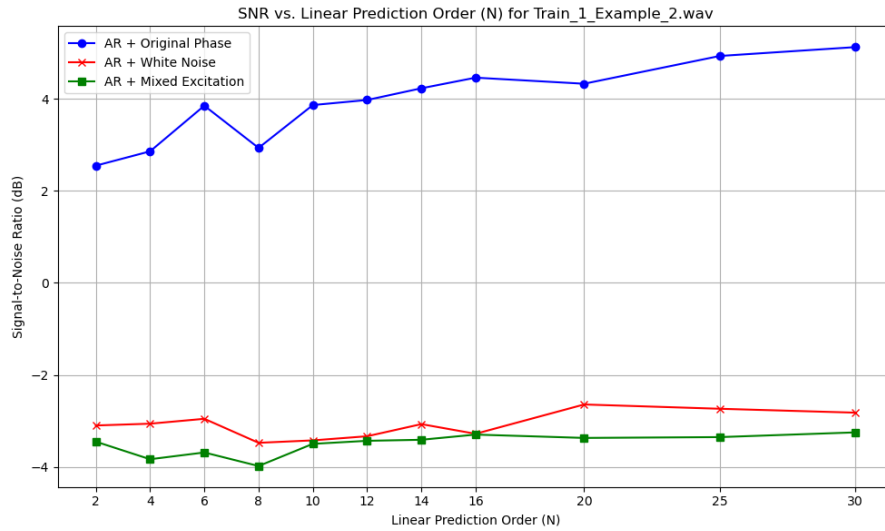


(b) SNR vs Level Order (N)

Figure 7: Train_1_Example_1.wav



(a) Waveform Comparison for $N = 10$



(b) SNR vs Level Order (N)

Figure 8: Train_1_Example_2.wav

Observation & Analysis: The reconstruction quality depends heavily on both the excitation source and the prediction order N , as can be seen in the from Fig: 5 to 8:

- **Excitation Source:** When the inverse LP filter formulation is driven solely by white noise excitation across all frames, the reconstructed speech sounds robotic and whispered, lacking natural pitch because the harmonic structure is missing. Using a pitch-based impulse train for voiced frames and white noise for unvoiced frames accurately simulates the source-filter model, yielding highly intelligible and natural-sounding speech.
- **Effect of N and Saturation:** As the LP order N increases, the AR model captures more

detail of the spectral envelope. However, the quality of reconstruction saturates after a relatively small choice of N (typically around 10 to 14, depending on the sampling rate).

- **Intuition:** A small N captures the macroscopic shape of the vocal tract (the formants). If N is chosen excessively large, the model begins to overfit, attempting to model the fine harmonic structure of the pitch rather than just the vocal tract envelope. This defeats the purpose of source-filter separation.

3.3 Part 6: Spectral Transform Linear Prediction (STLP)

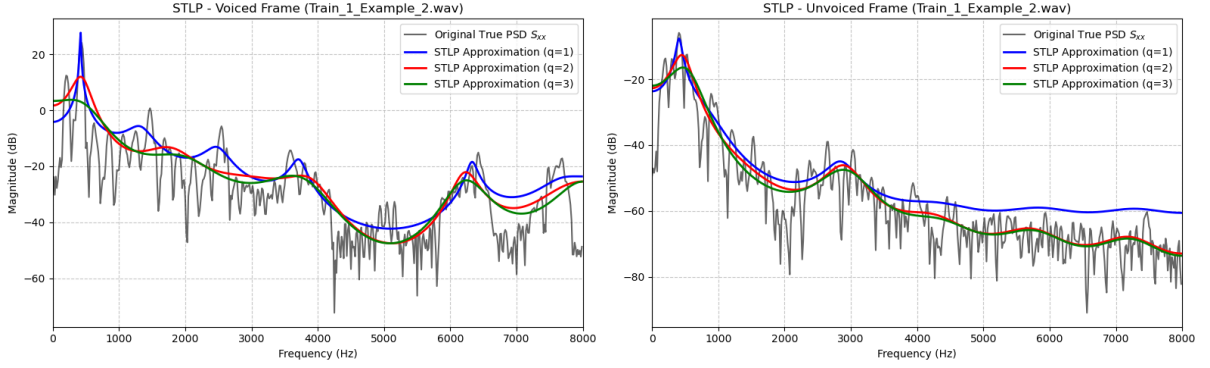


Figure 9: STLP approximations for a **voiced** & **unvoiced** frame with $q = 1, 2, 3$.

Observation & Analysis: Figures 9 demonstrate the effect of the parameter q on approximating the original PSD $S_{xx}(e^{j\omega})$.

- **Standard LP ($q = 1$):** The standard LP model effectively captures the spectral peaks (formants) but severely overestimates the spectral valleys. The mathematical formulation of standard LP inherently penalizes errors at high-energy regions more than low-energy regions.
- **STLP ($q > 1$):** By compressing the spectrum via $S_{xx}(e^{j\omega})^{\frac{1}{q}}$ before finding the AR coefficients, the dynamic range of the spectrum is reduced. As q increases from 1 to 3, the algorithm allocates more weighting to the valleys. When the spectrum is expanded back by raising it to q , the resulting envelope $\hat{S}_{xx}(e^{j\omega})^q$ provides a much tighter and more balanced fit across both the peaks and the deep valleys.
- **Voiced vs. Unvoiced:** This approximation improvement is highly pronounced in *voiced frames*, which naturally have a wide dynamic range with sharp, distinct formants. In contrast, *unvoiced frames* exhibit a flatter, noise-like spectrum; while STLP still operates effectively, the visual difference between $q = 1$ and higher values of q is less drastic because the original dynamic range is smaller.

4 Appendix: Source Code

```

1 # %%
2 import numpy as np
3 import scipy.signal as signal
4 import scipy.io.wavfile as wav
5 import scipy.linalg as linalg
6 import matplotlib.pyplot as plt
7 import os
8

```

```

9  # %% [markdown]
10 # ## Functions
11
12 # %%
13 def frame_signal(y, sr, frame_len_ms=25, hop_len_ms=10):
14     frame_length = int(sr * (frame_len_ms / 1000.0))
15     hop_length = int(sr * (hop_len_ms / 1000.0))
16     num_frames = 1 + int((len(y) - frame_length) / hop_length)
17     windowed_frames = np.zeros((num_frames, frame_length))
18     window = np.hamming(frame_length)
19
20     for i in range(num_frames):
21         start = i * hop_length
22         windowed_frames[i, :] = y[start:start + frame_length] * window
23
24     return windowed_frames, frame_length, hop_length
25
26 def compute_lpc_and_residual(windowed_frames, N):
27     num_frames, frame_length = windowed_frames.shape
28     lpc_coeffs = np.zeros((num_frames, N + 1))
29     residual_energy = np.zeros(num_frames)
30     residuals = np.zeros_like(windowed_frames)
31
32     for i in range(num_frames):
33         frame = windowed_frames[i, :]
34         r = np.correlate(frame, frame, mode='full')
35         r = r[len(frame)-1 : len(frame)-1 + N + 1]
36         R_matrix = linalg.toeplitz(r[:-1])
37         r_vector = r[1:]
38         R_matrix.flat[:N+1] += 1e-6
39         a_k = linalg.solve(R_matrix, r_vector)
40         a = np.concatenate(([1.0], -a_k))
41         lpc_coeffs[i, :] = a
42         res = signal.lfilter(a, [1.0], frame)
43         residuals[i, :] = res
44         residual_energy[i] = np.sum(res ** 2)
45
46     return lpc_coeffs, residuals, residual_energy
47
48 def reconstruct_ar_with_original_phase(windowed_frames, lpc_coeffs, N_fft=512):
49     num_frames, frame_length = windowed_frames.shape
50     recon_frames = np.zeros_like(windowed_frames)
51     for i in range(num_frames):
52         frame = windowed_frames[i, :]
53         frame_fft = np.fft.rfft(frame, n=N_fft)
54         original_phase = np.angle(frame_fft)
55         w, h = signal.freqz([1.0], lpc_coeffs[i, :], worN=N_fft//2 + 1,
56 include_nyquist=True)
57         ar_magnitude = np.abs(h)
58         gain = np.sqrt(np.sum(frame**2) / (np.sum(np.fft.irfft(ar_magnitude)**2)
59 + 1e-10))
60         ar_magnitude *= gain
61         recon_fft = ar_magnitude * np.exp(1j * original_phase)
62         recon_frames[i, :] = np.fft.irfft(recon_fft, n=N_fft)[:frame_length]
63     return recon_frames
64
65 def synthesize_artificial(windowed_frames, lpc_coeffs, mode="white", is_voiced=
66 None, pitch=None):
67     num_frames, frame_length = windowed_frames.shape
68     synth_frames = np.zeros_like(windowed_frames)
69     for i in range(num_frames):
70         frame_energy = np.sum(windowed_frames[i, :]**2)
71         if mode == "white":

```

```

69         exc = np.random.randn(frame_length)
70     else: # mixed
71         if is_voiced[i]:
72             exc = np.zeros(frame_length)
73             exc[:,pitch[i]] = 1.0
74         else:
75             exc = np.random.randn(frame_length) * 0.1
76
77     raw_synth = signal.lfilter([1.0], lpc_coeffs[i, :], exc)
78     synth_energy = np.sum(raw_synth**2) + 1e-10
79     # Energy matching to prevent amplitude explosion
80     synth_frames[i, :] = raw_synth * np.sqrt(frame_energy / synth_energy)
81     return synth_frames
82
83 def overlap_add(frames, hop_length):
84     num_frames, frame_length = frames.shape
85     total_length = (num_frames - 1) * hop_length + frame_length
86     out_signal = np.zeros(total_length)
87     for i in range(num_frames):
88         start = i * hop_length
89         out_signal[start:start + frame_length] += frames[i, :]
90     return out_signal
91
92 def estimate_voicing_and_pitch(frames, sr):
93     num_frames, frame_len = frames.shape
94     is_voiced = np.zeros(num_frames, dtype=bool)
95     pitch_periods = np.zeros(num_frames, dtype=int)
96     energies = np.sum(frames**2, axis=1)
97     energy_threshold = np.mean(energies) * 0.1
98     for i in range(num_frames):
99         if energies[i] > energy_threshold:
100             r = np.correlate(frames[i], frames[i], mode='full')
101             r = r[frame_len-1:]
102             min_lag, max_lag = int(sr * 0.002), int(sr * 0.020)
103             if max_lag < len(r):
104                 peak_lag = np.argmax(r[min_lag:max_lag]) + min_lag
105                 if r[peak_lag] > 0.3 * r[0]:
106                     is_voiced[i] = True
107                     pitch_periods[i] = peak_lag
108     return is_voiced, pitch_periods
109
110 def calculate_snr(original, reconstructed):
111     min_len = min(len(original), len(reconstructed))
112     orig_sliced = original[:min_len]
113     recon_sliced = reconstructed[:min_len]
114     signal_energy = np.sum(orig_sliced**2)
115     noise_energy = np.sum((orig_sliced - recon_sliced)**2)
116     return 10 * np.log10(signal_energy / (noise_energy + 1e-10))
117
118 def plot_waveforms_N10(filename, y, sig_phase, sig_noise, sig_mix, sr, out_dir):
119     min_len = min(len(y), len(sig_phase), len(sig_noise), len(sig_mix))
120     t_axis = np.arange(min_len) / sr
121     fig, axs = plt.subplots(4, 1, figsize=(12, 10), sharex=True)
122     axs[0].plot(t_axis, y[:min_len], color='black')
123     axs[0].set_title(f'Original Waveform: {filename}')
124     axs[1].plot(t_axis, sig_phase[:min_len], color='blue')
125     axs[1].set_title('Reconstruction: AR + Original Phase (N=10)')
126     axs[2].plot(t_axis, sig_noise[:min_len], color='red')
127     axs[2].set_title('Reconstruction: AR + White Noise (N=10)')
128     axs[3].plot(t_axis, sig_mix[:min_len], color='green')
129     axs[3].set_title('Reconstruction: AR + Mixed Excitation (N=10)')
130     axs[3].set_xlabel('Time (seconds)')
131     for ax in axs:

```

```

132         ax.grid(True)
133         ax.set_ylabel('Amplitude')
134
135     plt.tight_layout()
136     plt.savefig(os.path.join(out_dir, f'Waveforms_N10_{filename.replace(".wav",
137     plt.show()
138     plt.close(fig)
139
140 def plot_snr_analysis(filename, N_values, snr_phase, snr_noise, snr_mix, out_dir
141 ):
142     plt.figure(figsize=(10, 6))
143     plt.plot(N_values, snr_phase, marker='o', color='blue', label='AR + Original
144     Phase')
145     plt.plot(N_values, snr_noise, marker='x', color='red', label='AR + White
146     Noise')
147     plt.plot(N_values, snr_mix, marker='s', color='green', label='AR + Mixed
148     Excitation')
149
150     plt.title(f'SNR vs. Linear Prediction Order (N) for {filename}')
151     plt.xlabel('Linear Prediction Order (N)')
152     plt.ylabel('Signal-to-Noise Ratio (dB)')
153     plt.xticks(N_values)
154     plt.legend()
155     plt.grid(True)
156     plt.tight_layout()
157     plt.savefig(os.path.join(out_dir, f'SNR_vs_N_{filename.replace(".wav", "").}
158     png'))
159     plt.show()
160     plt.close()
161
162 # %% [markdown]
163 # ## Q1 Linear Prediction
164
165 # %%
166 data_dir = 'data'
167 files = os.listdir(data_dir)
168 files
169
170 # %% [markdown]
171 # #### Q1.a)
172
173 # %%
174 N_part_a = 10
175
176 # %%
177 for file_name in files:
178     file_path = os.path.join(data_dir, file_name)
179     sr, y = wav.read(file_path)
180     y = y.astype(np.float64)
181     y = y / (np.max(np.abs(y)) + 1e-10)
182     file_name = file_name.replace('.wav', '')
183
184     windowed_frames, frame_len, hop_len = frame_signal(y, sr, 25, 10)
185     lpc_coeffs, residuals, residual_energy = compute_lpc_and_residual(
186     windowed_frames, N_part_a)
187     original_energy = np.sum(windowed_frames**2, axis=1)
188
189     # For Visualization convenience
190     original_energy_db = 10 * np.log10(original_energy + 1e-10)
191     residual_energy_db = 10 * np.log10(residual_energy + 1e-10)
192
193     t_wave = np.arange(len(y)) / sr

```

```

188     t_frames = (np.arange(len(original_energy)) * hop_len + (frame_len / 2)) /
189     sr
190
191     fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8), sharex=True)
192     ax1.plot(t_wave, y, color='slategray', alpha=0.9)
193     ax1.set_title(f'Raw Speech Waveform for {file_name + '.wav'}')
194     ax1.set_ylabel('Amplitude')
195     ax1.grid(True)
196
197     # Bottom Plot: Original Energy vs Residual Energy
198     ax2.plot(t_frames, original_energy_db, label='Original Frame Energy', color=
199     'blue')
200     ax2.plot(t_frames, residual_energy_db, label=f'Residual Energy (N={N_part_a
201     })', color='red', linestyle='--')
202     ax2.set_title(f'Original Energy vs. Residual Energy (Order N={N_part_a})')
203     ax2.set_xlabel('Time (seconds)')
204     ax2.set_ylabel('Energy (dB)')
205     ax2.legend()
206     ax2.grid(True)
207
208     plt.tight_layout()
209     plot_path = os.path.join('output', f'Plot_PartA_{file_name}.png')
210     plt.savefig(plot_path)
211     plt.show()
212     plt.close()
213
214 # %% [markdown]
215 # ### Q1.b)
216
217 # %%
218 for file_name in files:
219     file_path = os.path.join(data_dir, file_name)
220     sr, y = wav.read(file_path)
221     y = y.astype(np.float64) / (np.max(np.abs(y)) + 1e-10)
222     windowed_frames, frame_len, hop_len = frame_signal(y, sr, 25, 10)
223     is_voiced, pitch = estimate_voicing_and_pitch(windowed_frames, sr)
224
225     lpc_N10, _, _ = compute_lpc_and_residual(windowed_frames, 10)
226     frames_phase = reconstruct_ar_with_original_phase(windowed_frames, lpc_N10)
227     frames_noise = synthesize_artificial(windowed_frames, lpc_N10, mode="white")
228     frames_mix = synthesize_artificial(windowed_frames, lpc_N10, mode="mixed",
229     is_voiced=is_voiced, pitch=pitch)
230
231     sig_phase = overlap_add(frames_phase, hop_len)
232     sig_noise = overlap_add(frames_noise, hop_len)
233     sig_mix = overlap_add(frames_mix, hop_len)
234
235     plot_waveforms_N10(file_name, y, sig_phase, sig_noise, sig_mix, sr, 'output'
236     )
237
238 # %%
239 N_values_to_test = [2, 4, 6, 8, 10, 12, 14, 16, 20, 25, 30]
240
241 # %%
242 for file_name in files:
243     file_path = os.path.join(data_dir, file_name)
244     sr, y = wav.read(file_path)
245     y = y.astype(np.float64) / (np.max(np.abs(y)) + 1e-10)
246     windowed_frames, frame_len, hop_len = frame_signal(y, sr, 25, 10)
247     is_voiced, pitch = estimate_voicing_and_pitch(windowed_frames, sr)
248
249     snr_phase_list, snr_noise_list, snr_mix_list = [], [], []

```

```

246
247     for N in N_values_to_test:
248         lpc_N, _, _ = compute_lpc_and_residual(windowed_frames, N)
249
250         sig_p = overlap_add(reconstruct_ar_with_original_phase(windowed_frames,
251 lpc_N), hop_len)
252         sig_n = overlap_add(synthesize_artificial(windowed_frames, lpc_N, mode="
253 white"), hop_len)
254         sig_m = overlap_add(synthesize_artificial(windowed_frames, lpc_N, mode="
255 mixed", is_voiced=is_voiced, pitch=pitch), hop_len)
256
257         snr_phase_list.append(calculate_snr(y, sig_p))
258         snr_noise_list.append(calculate_snr(y, sig_n))
259         snr_mix_list.append(calculate_snr(y, sig_m))
260
261     plot_snr_analysis(file_name, N_values_to_test, snr_phase_list,
262 snr_noise_list, snr_mix_list, 'output')
263
264 # %% [markdown]
265 # ## Q2 Spectral Transform Linear Prediction
266
267 # %%
268 def compute_stlp_and_plot(voiced_frame, unvoiced_frame, p_order, q_values, sr,
269 file_name=""):
270     fig, axes = plt.subplots(1, 2, figsize=(16, 5))
271     for idx in range(2):
272         if idx == 0:
273             frame = voiced_frame
274             title = f"Voiced Frame ({file_name})"
275         else:
276             frame = unvoiced_frame
277             title = f"Unvoiced Frame ({file_name})"
278         N_fft = 1024
279         X = np.fft.fft(frame, n=N_fft)
280         S_xx = np.abs(X)**2
281         S_xx = np.maximum(S_xx, 1e-10)
282         freqs = np.fft.fftfreq(N_fft, 1/sr)
283         half_idx = N_fft // 2
284         axes[idx].plot(freqs[:half_idx], 10 * np.log10(S_xx[:half_idx]), color='
285 black', label='Original True PSD $S_{xx}$', alpha=0.6, linewidth=1.5)
286         colors = ['blue', 'red', 'green']
287
288         for i, q in enumerate(q_values):
289             S_xx_compressed = S_xx * (1.0 / q)
290             r_q = np.real(np.fft.ifft(S_xx_compressed))
291             r_q_trunc = r_q[:p_order + 1]
292             R_matrix = linalg.toeplitz(r_q_trunc[:-1])
293             r_vector = r_q_trunc[1:]
294             R_matrix.flat[:p_order+1] += 1e-6
295             a_k = linalg.solve(R_matrix, r_vector)
296             a_coeffs = np.concatenate([1.0, -a_k])
297             gain_compressed = r_q_trunc[0] - np.dot(a_k, r_vector)
298             gain_compressed = max(gain_compressed, 1e-10)
299             _, h = signal.freqz([1.0], a_coeffs, worN=N_fft, whole=True)
300             S_hat_compressed = gain_compressed * (np.abs(h)**2)
301             S_hat_expanded = S_hat_compressed * q
302             axes[idx].plot(freqs[:half_idx], 10 * np.log10(S_hat_expanded[:
303 half_idx]),
304                             color=colors[i % len(colors)],
305                             label=f'STLP Approximation (q={q})', linewidth=2)
306
307         axes[idx].set_title(f'STLP - {title}')
308         axes[idx].set_xlabel('Frequency (Hz)')

```

```

302     axes[idx].set_ylabel('Magnitude (dB)')
303     axes[idx].set_xlim([0, sr/2])
304     axes[idx].grid(True, linestyle='--', alpha=0.7)
305     axes[idx].legend()
306     plt.tight_layout()
307     plt.savefig(os.path.join('output', f'STLP_voice_vs_unvoice_{file_name.
replace(".wav", "").}.png'))
308     plt.show()
309
310 # %%
311 file_path = os.path.join(data_dir, files[0])
312
313 sr, y = wav.read(file_path)
314 if y.dtype == np.int16:
315     y = y.astype(np.float32) / np.iinfo(np.int16).max
316 elif y.dtype == np.int32:
317     y = y.astype(np.float32) / np.iinfo(np.int32).max
318 else:
319     y = y.astype(np.float32)
320 if len(y.shape) > 1:
321     y = np.mean(y, axis=1)
322
323 windowed_frames, frame_length, hop_length = frame_signal(y, sr, frame_len_ms=30,
hop_len_ms=10)
324 is_voiced, pitch_periods = estimate_voicing_and_pitch(windowed_frames, sr)
325
326 voiced_indices = np.where(is_voiced == True)[0]
327 unvoiced_indices = np.where(is_voiced == False)[0]
328
329 if len(voiced_indices) > 0 and len(unvoiced_indices) > 0:
330     voiced_frame = windowed_frames[voiced_indices[len(voiced_indices)//2]]
331     unvoiced_frame = windowed_frames[unvoiced_indices[len(unvoiced_indices)//2]]
332
333     p_order = 12
334     q_values = [1, 2, 3]
335
336     compute_stlp_and_plot(voiced_frame, unvoiced_frame, p_order, q_values, sr,
file_name=file_name)
337
338 # %%

```

Listing 1: TejashMore27077.py